

Claude로 캐글 입문하기



안드로이드 개발자가 6주 동안 캐글로 시도해본 것들

캐글이란 무엇인가

전 세계 사람들이 같은 데이터로 AI 모델을 만들고 점수로 겨루는 곳

캐글 = 데이터 + 문제 + 경쟁

회사·연구자가 데이터를 올리고

"이걸로 X를 예측해주세요"라는 문제를 낸다.

누구나 AI 모델을 만들어 점수로 경쟁한다.

2010년 설립 → **2017년 구글 인수**. 가입자 1,500만 명 이상. 무료로 참가 가능.

생각보다 푸는 문제가 다양하다

사진·영상 (Image)

폐 X-ray 폐렴 감지 / 위성 사진 홍수 피해 탐지 / 물체 감지 / 의료 이미지 판독

소리 인식 (Audio)

새 울음소리 234종 구분 — **오늘 사례** / 음악 장르 분류 / 음성 인식

강화학습·시뮬레이션

AI가 미로 탈출 / 우주선 궤도 조종 / 체스·바둑 같은 전략 게임

텍스트·번역 (Text/NLP)

재난 트윗 진위 판별 / 리뷰 감성 분석 / 고대 문서 기계번역 / 뉴스 요약

수치 예측 (Tabular)

집값·매출 예측 / 고객 이탈률 / 스포츠 경기 결과 / NFL 빅데이터 분석

과학·수학·추론

RNA 3D 구조 예측 / 단백질 기능 분류 / AI 수학올림피아드 / ARC Prize (AGI 추론, 상금 \$850만)

우승하면 상금도 있다

\$수만~수십만

Featured 대회 기준 (규모마다 다름)

- **Featured 대회** — 회사·기관이 후원. 상금이 크다
- **Playground / Knowledge** — 상금 없지만 입문용으로 좋다. 매달 새 대회
- **Code Competition** — 노트북 자체로 제출. 실행 시간 제한 있음

점수 체계

자주 쓰는 점수 3가지

정확도 (Accuracy)

100개 중 몇 개 맞혔나

0~1, 높을수록 좋음

"이 사진은 고양이다/개다" 같은 단순 분류

AUC

양성/음성을 얼마나 잘 구분하는 능력 점수

0.5 = 찍기, 1.0 = 완벽

높을수록 좋음 (0~1)

(Area Under ROC Curve)

RMSE

예측값이 정답에서 평균 얼마나 틀렸나

낮을수록 좋음 (0~∞)

집값 같은 숫자 예측에 씬

(Root Mean Square Error)

Public 점수 vs Private 점수

Public 점수

대회 진행 중

전체 시험 문제 중 일부(약 30%)만 미리 채점해서 보여주는 점수.

대회 기간 내내 표시. 다른 사람 점수도 볼 수 있음.

Private 점수

대회 종료 후

나머지(약 70%)로 채점한 최종 점수. 종료 시점에 공개.

이게 진짜 등수. Public 점수와 순위가 크게 달라지기도 한다.

왜 둘로 나눌까? — Public만 보고 과적합(그 30%에만 맞도록 끼워맞추기)하는 걸 막기 위해. 일일 제출은 보통 5회 제한.

노트북 — 코드와 결과가 한 곳에

노트북 = 코드 + 실행 결과 + 글이 한 페이지에

구글 독스인데 코드가 같이 들어 있고, 그 자리에서 실행되어 그래프·이미지가 바로 나오는 문서.

- 위에서 아래로 셀(cell) 단위로 실행. 결과가 그 자리에 즉시 표시
- 캐글에서 무료로 GPU 달린 서버 제공 — 주당 약 30시간 (정책 변동 가능)
- .ipynb 파일이라 부른다 (Jupyter Notebook)

오디오 파일을 불러옵니다

```
y, sr = librosa.load("birdsong.wav", sr=32000)
```

로딩 완료 — sr=32000Hz, 5.0초

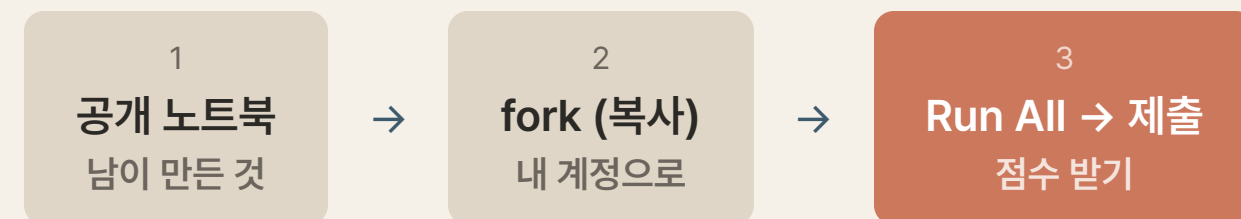
멜스펙트로그램으로 변환

```
mel = librosa.feature.melspectrogram(y, sr=sr)
```

mel.shape == (128, 156) — 소리가 이미지가 됨

남의 노트북을 복사해서 바로 쓴다 (fork)

공개 노트북(Public Notebook)은 누군가가 만들어서 공개한 코드. 캐글에는 수십만 개가 있습니다.



코드 한 줄 못 짜도 fork → Run All → Submit까지 가능.

점수 잘 나오는 노트북을 fork → 한 줄씩 바꿔보며 점수 올리기 ← 입문 패턴

Kaggle CLI — 딱 두 가지만 직접 한다

직접 해야 하는 것 (최초 1회)

```
# 1. CLI 설치
$ pip install kaggle

# 2. API 키 받기
# kaggle.com → 우측 프로필 사진
# → Settings → API 섹션
# → "Create New API Token" 클릭
# → kaggle.json 파일 다운로드됨

# 3. 키 파일 배치
$ mkdir -p ~/.kaggle
$ mv kaggle.json ~/.kaggle/
$ chmod 600 ~/.kaggle/kaggle.json
```

이후엔 Claude한테 맡기면 됨.

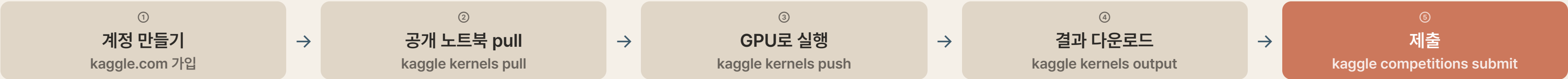
CLI로 할 수 있는 것들

- ▶ 공개 노트북 다운로드 (kernels pull)
- ▶ 노트북 캐글 서버에 업로드·실행 (kernels push)
- ▶ 실행 결과 다운로드 (kernels output)
- ▶ 대회에 제출 (competitions submit)
- ▶ 리더보드 확인 (competitions leaderboard)
- ▶ 데이터셋 다운로드 (datasets download)
- ▶ 노트북 실행 상태 확인 (kernels status)

핵심: 이 모든 게 명령어라서 Claude가 대신 실행할 수 있다.

어떻게 참가하나

30분이면 첫 제출 가능



전부 터미널 명령어로 가능. 웹사이트 안 열어도 됨. Claude가 이 흐름을 자동화한다.

데이터 분석 — 어떤 순서로 하나

ML 프로젝트, 이 순서로 흘러간다



실제로 시간을 가장 많이 쓰는 곳

80% 데이터 수집 · 전처리 · 피처 엔지니어링

20% 모델링 · 튜닝 · 평가

캐글에서 다른 점

- ✓ 데이터는 대회에서 제공 — 수집 불필요
- ✓ 문제 정의도 완료 — 바로 분석부터
- ✓ 평가 기준도 정해짐 — 리더보드로 확인

각 단계에서 실제로 뭘 하나

🎯 문제 정의

뭘 예측·분류할지 결정
분류 / 수치예측 / 이상탐지 중 선택
평가 지표 (AUC, RMSE 등) 확정
★ 잘못 정의하면 모든 게 헛수고

📦 데이터 수집

DB 쿼리, API 호출, CSV
웹 크롤링, 로그, 센서 데이터
정답 라벨링 (지도학습 필수)
캐글: 대회가 제공 → 수집 불필요

🔍 EDA (탐색적 분석)

분포 시각화 — 히스토그램·박스플롯
결측값·이상치 위치 파악
변수 간 상관관계 확인
데이터 이해 없이 모델 못 만든다

🔧 전처리

결측값 → 평균·중앙값으로 채우기
이상치 제거 또는 상한 캡핑
정규화 (0~1) / 표준화 (평균 0)
문자 → 숫자 변환 (원-핫 인코딩)

🔧 피처 엔지니어링

생년월일 → 나이 / 가입일 → 이용기간
키 + 몸무게 → BMI
날짜 → 요일·월·공휴일 여부
★ 도메인 지식이 가장 중요한 단계

🤖 모델링

표 데이터 → XGBoost, LightGBM
이미지·소리 → CNN, EfficientNet
텍스트 → Transformer, BERT
하이퍼파라미터 튜닝 + 교차검증(CV)

📁 평가 & 제출

테스트셋으로 최종 성능 측정
앙상블 — 여러 모델 예측 합치기

캐글: 리더보드에 제출 → 점수 확인
실무: API 서버 배포, A/B 테스트

데이터를 모델이 먹을 수 있게 만든다

🔧 전처리 (Preprocessing)

- 결측값 처리 — 빈칸을 평균/중앙값으로 채우거나 행 제거
예: 나이 데이터에 빈 칸 → 평균 나이로 채움
- 이상치 제거 — 말도 안 되는 값 걸러내기
예: 나이가 999살 → 제거
- 정규화/표준화 — 숫자 범위를 같은 스케일로
예: 키(170cm)와 몸무게(70kg)를 0~1 범위로
- 범주형 인코딩 — 문자 → 숫자
예: "남/여" → 0/1, 지역명 → 원-핫 인코딩
- 소리 → 이미지 변환 (BirdCLEF)
소리 파형 → 멜-스펙트로그램(주파수 이미지) → CNN 입력

🔧 피처 엔지니어링

- 새 변수 만들기
생년월일 → 나이 / 가입일 → 사용 기간
- 조합 변수
키 + 몸무게 → BMI = 체중/키²
- 시간 분해
날짜 → 요일, 월, 공휴일 여부
- 집계 통계
고객별 최근 30일 평균 구매금액

🤖 모델링 선택 기준

- | | |
|--------------------------------------|------------------------------------|
| 표 형태 데이터
XGBoost, LightGBM | 이미지/소리
CNN, EfficientNet |
| 텍스트/번역
Transformer, BERT | 무난한 시작
RandomForest |

Claude로 뭔 하나

"Claude, 이 코드 한국어로 설명해줘"

fork한 노트북은 영어 + 줄임말 + ML 용어 범벅.

Claude에게 셀 단위로 붙여넣고 물으면 — 줄별 한국어 해설. 이 변수가 뭔지, 왜 이 함수를 쓰는지, 이 셀의 결과가 뭘 의미하는지.

나

이 코드 무슨 뜻인지 알려줘:

```
mel = librosa.feature.melspectrogram(y,  
sr=32000)  
# mel.shape == (128, 156)
```

소리를 이미지로 바꾸는 코드입니다.

y 는 새 울음소리 wav. 이걸 멜스펙트로그램(주파수×시간 그림)으로 변환합니다.

결과 (128, 156) — 128은 주파수 구간, 156은 시간 구간.

소리를 이미지로 바꾸면 이미지 분류 모델(ResNet, ConvNeXt 등)을 그대로 쓸 수 있어 효과적입니다.

Claude

"에러 메시지 그대로 붙여넣기 → 해결책"

에러 메시지 그대로 붙여넣으면 끝. 한 줄도 다듬을 필요 없어요.

사용자: 이 에러가 났어요

```
RuntimeError: CUDA out of memory.  
Tried to allocate 2.50 GiB  
(GPU 0; 15.78 GiB total capacity;  
14.12 GiB already allocated)
```

Claude 답변

GPU 메모리 부족입니다. 세 가지 시도:

1. **batch size**를 **32 → 16**으로 줄이기 (즉시 효과)

2. **mixed precision(fp16)** 사용 — 메모리 절반:

```
accelerator = Accelerator(mixed_precision="fp16")
```

3. **gradient accumulation** — 작은 batch로 큰 batch 효과

※ 단어가 어려워도 OK — 핵심은 "Claude가 해결책을 줬다"는 흐름입니다.

"점수 올리려면 뭘 바꿔야 해?"

현재 점수와 노트북을 보여주고 묻는다. Claude가 시도해볼 만한 제안을 근거와 함께 준다.

예시 대화

현재 0.912. 더 올리고 싶어.

세 가지 추천:

- **데이터 증강 (augmentation)** — 같은 새 소리를 살짝 변형해서 더 많이 보여주기. 시간 늘리기, 잡음 추가, 음높이 이동 등. **가장 ROI 큼.**
augmentation = 데이터 보강 (외부 데이터 없이 기존 데이터를 변형)
- **학습률 cosine 스케줄** — 학습률을 전체적으로 부드럽게 줄여 후반 미세 조정. 한 줄 변경.
- **여러 모델 앙상블** — ResNet + EfficientNet + ConvNeXt 가중평균.

자주 나오는 ML 용어 정리

이제 실전입니다. 모르는 용어가 나오면 여기로 돌아오세요.

용어	무슨 뜻인지
딥러닝 (Deep Learning)	뇌의 뉴런 구조를 흉내낸 여러 겹의 레이어로 만든 AI — 이미지·소리·텍스트처럼 복잡한 데이터에 강함
GPU	병렬 연산 특화 칩 — 딥러닝 학습에 CPU보다 수십~수백 배 빠름. 자세한 내용은 Slide 20 참고.
train set (학습 데이터)	모델에게 보여주는 문제+정답 묶음 — 여기서 패턴을 학습함
validation set (검증 데이터 / holdout)	학습에 안 쓰고 따로 빼둔 데이터 — 학습 중 성능 확인용. 일반적으로 전체의 10~20%
test set (테스트 데이터)	캐글에서 제출하면 서버가 채점에 쓰는 데이터 — 정답을 볼 수 없음. 이게 LB 점수
모델 (Model)	입력을 넣으면 예측값이 나오는 수학 함수
학습 (Training)	데이터를 반복해서 보여주며 모델이 정답에 가까워지도록 조정하는 과정
추론 (Inference)	학습이 끝난 모델을 실제 데이터에 적용해 예측값 뽑기
앙상블 (Ensemble)	여러 모델의 예측을 합쳐 더 안정적인 결과 만들기
CV (교차 검증)	내가 가진 데이터를 쪼개 스스로 점수를 매기는 방식
LB (리더보드)	캐글 서버에 제출해서 받는 공식 점수
CV-LB gap	내 점수와 공식 점수의 차이 — 크면 모델이 학습 데이터만 외운 것
overfitting (과적합)	학습 데이터에는 잘 맞지만 새 데이터에는 못 맞추는 상태
dropout	학습 중 일부 뉴런을 무작위로 끄기 — 외워버리는 것 방지
augmentation (데이터 보강)	같은 데이터를 살짝 변형해 더 많이 보여주기 (잡음 추가, 뒤집기 등)
스펙트로그램	소리를 시간 × 주파수 축으로 표현한 이미지
ResNet / EfficientNet / ConvNeXt	이미지를 잘 인식하는 딥러닝 모델 구조 이름 (종류가 여러 가지 있음)
batch size (배치 크기)	한 번에 모델에 넣는 데이터 개수 — 클수록 메모리 더 필요
mixed precision (fp16)	16비트 연산 방식 — 메모리 절반, 속도 빠름
gradient accumulation	작은 배치를 여러 번 모아 큰 배치처럼 학습 — 메모리 부족할 때 쓰는 우회법

fold blend — 5명한테 따로 묻고 평균 내기

데이터를 5등분 → 5번 학습 → 5명의 답을 평균.

핵심 직감

혼자 짚지 말고 다섯 명한테 따로 물어본 다음 답을 평균.
한 명이 틀려도 나머지가 보정해줌.

fold = 접어서 나눈 한 조각. **1-fold** = 1명만, **5-fold** = 5명 평균.

데이터 1000개를 5등분 → 5명이 다른 조각으로 학습

모델	학습한 FOLD	검증 FOLD
A	② ③ ④ ⑤	①
B	① ③ ④ ⑤	②
C	① ② ④ ⑤	③
D	① ② ③ ⑤	④
E	① ② ③ ④	⑤

제출 시: 5명의 예측을 평균

A: 0.70 · B: 0.80 · C: 0.60 · D: 0.75 · E: 0.85 → **평균 0.74**

앙상블 — 다른 종류 모델들의 답을 가중평균

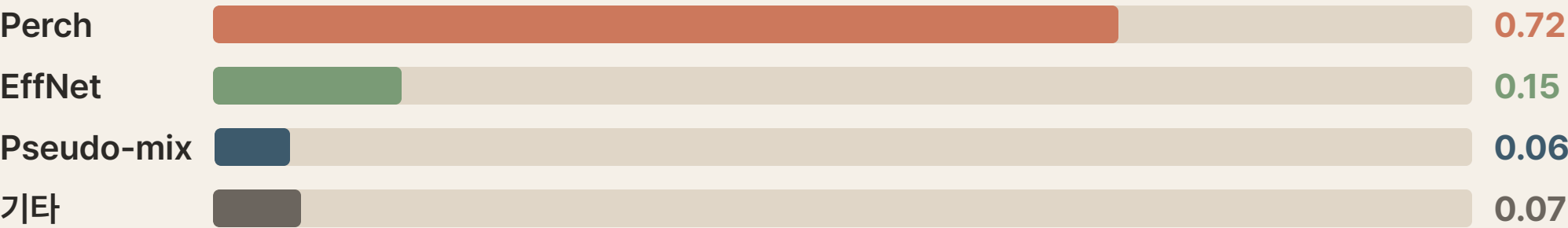
fold blend는 같은 종류 5명, **앙상블**은 서로 다른 종류 N명.

모델마다 보는 각도가 달라서 약점이 보완된다.
한 모델이 헛갈리는 새 소리를 다른 모델은 잘 맞히기도 함.

핵심 직감

"의사 1명한테만 묻지 말고, 내과 + 영상의학 + 외과한테 다 묻고 종합 판단."

BirdCLEF 최종 가중치 (합 = 1.0)



Perch 단독으로도 강하지만, EffNet·Pseudo-mix가 약 0.022 점수 추가 기여.

Perch — Google이 만든 새소리 전문 AI

Google Research가 **9,736종 새소리**로 미리 학습시켜 무료로 공개한 모델.

- 입력: 5초 wav
- 출력: 9,736종 확률 + embedding 벡터
- 우리는 그중 234종만 추려서 사용

핵심 직감

"새소리 박사학위 따놓은 사람이 무료로 공개해놨음. 처음부터 만들지 말고 그 사람한테 물어보기."



BirdCLEF 점수 기여도

전체의 70%+ — Perch 없으면 0.912조차 안 나옴.

처음부터 만들면 GPU 수백 대 × 몇 달. 공개 모델 활용은 캐글의 기본기.

Embedding — 데이터를 숫자 벡터로 요약한 것

이미지·소리·텍스트를 **1024개쯤 되는 숫자 배열**로 변환한 것.

- 비슷한 데이터는 비슷한 숫자가 됨
- 컴퓨터가 거리·유사도를 계산하기 쉬워짐
- Perch는 wav를 **1280차원 벡터**로 요약 → 그 벡터로 분류기 학습

핵심 직감

"사물의 DNA처럼 숫자로 요약된 지문. 같은 종은 비슷한 지문이 나옴."

예: 새소리 wav → embedding → 234종 분류

까마귀 소리 5초 wav

`vec_a = perch.embed(wav_a)`

[0.12, -0.83, 0.41, ..., 0.07] (1280개)

다른 까마귀 소리

`vec_b = perch.embed(wav_b)`

[0.10, -0.80, 0.38, ..., 0.09] ← 비슷한 숫자

참새 소리

[-0.55, 0.42, -0.18, ..., 0.91] ← 멀리 떨어진 숫자

분류기는 1280숫자만 보고 "이건 까마귀" 판단. 원본 wav를 다시 볼 필요 없음.

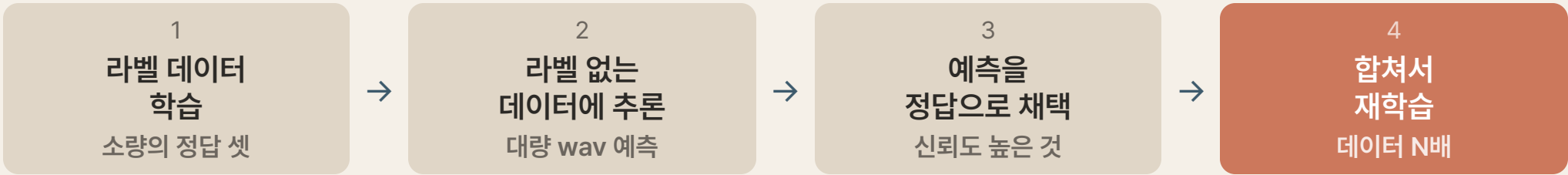
Pseudo-label — AI가 매긴 답을 정답처럼 다시 학습

라벨 없는 데이터에 모델이 답을 매기고, 그 답을 정답처럼 써서 또 학습시키는 것.

- 라벨 있는 데이터 부족할 때 효과적
- 잘못된 답까지 학습할 위험 → 신뢰도 높은 것만 선별
- BirdCLEF에서 **0.933 → 0.934 천장 돌파의 핵심**

핵심 직감

"선생님이 푼 모의고사 답안을 학생에게 정답지로 줘서 한 번 더 공부시키기."



BirdCLEF 결과 — pseudo-mix 기법

기존 0.933 → **0.934 ✓** (4-fold 앙상블 시)

GPU가 필요한 경우 vs CPU로 충분한 경우

CPU로 충분

빠른 연산보다 많은 규칙 탐색이 중요한 경우

- **테이블 데이터 예측 (Tabular)**
집값·고객이탈·매출 예측. LightGBM·XGBoost 같은 트리 기반 모델 사용. 학습 수 분~수십 분.
- **스포츠·금융 통계 분석**
NFL Big Data Bowl, 시계열 예측 등 수치 위주 대회

예시 대회: Playground Series (매달 수치 예측), March Mania (농구 결과 예측)

GPU 거의 필수

수백만 번 연산을 동시에 해야 하는 딥러닝

- **이미지·영상 분류**
의료 이미지, 위성 사진, 물체 감지. GPU 없으면 CPU 대비 수십 배 느림.
- **소리 인식 (Audio)**
BirdCLEF처럼 wav → 스펙트로그램 이미지로 변환 후 딥러닝 학습
- **텍스트·번역 (NLP)**
BERT·GPT 계열 언어 모델. 파라미터 수억 개 → GPU 필수
- **수학·추론 (ARC Prize 등)**
대형 언어 모델 파인튜닝

캐글 무료 GPU: T4 / P100 — 주당 30시간 제공. 비용 0원.

Apple Silicon — Mac에서도 학습 가능

M1·M2·M3·M4 칩 안에는 **CPU + GPU + Neural Engine(NPU)**가 한 칩에 통합되어 있다.

- **NPU = Neural Processing Unit** — AI 연산 전용 회로. Apple은 "Neural Engine"이라 부름. 16코어.
- **Apple GPU** — Metal API로 접근. NVIDIA CUDA 대신 PyTorch가 `device="mps"`로 사용.
- Neural Engine(NPU)은 PyTorch에서 직접 못 씀 → CoreML 변환 필요. 학습은 보통 GPU(MPS) 백엔드.

핵심 직감

"NVIDIA GPU 없어도 맥북 한 대로 학습이 된다. 무겁지 않은 모델이면 캐글 GPU 안 빌려도 돌아감."

왜 GPU 대체가 되나

NVIDIA GPU

CUDA 코어 수천 개로 행렬 연산 동시 처리. 표준.
수만 원~수천만 원. 별도 장비.

Apple GPU (MPS)

Apple GPU 코어로 같은 행렬 연산. PyTorch가 Metal로 변환.
맥북 한 대에 다 들어있음.

BirdCLEF 사례

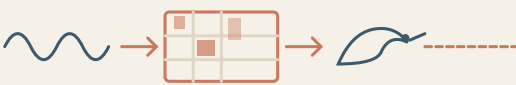
EfficientNet 5-fold 학습을 **M1 맥북 MPS**로 직접 진행. 캐글 GPU는 추론·양상블에만 사용.

한계: NVIDIA보다 일반적으로 느림. 일부 연산(예: fp16 자동 변환) 미지원. 큰 LLM 학습은 어려움.

실전 — BirdCLEF 2026

BirdCLEF 2026 — 새 소리로 234종 구분하기

- 입력 — 30초짜리 wav 녹음
- 출력 — 234종 중 어떤 새가 울었는지 (확률)
- 모델 — AI가 예측하는 수학 함수. 입력 → 출력을 학습으로 만든 것
- 방법 — 소리를 멜스펙트로그램(주파수×시간 그림)으로 바꿔 이미지 인식 모델로 분류
- 핵심 모델: **Perch** — Google이 만든 새 소리 전용 AI. 공개되어 있어서 가져다 씀. 9,736종 학습됨.
+ 직접 학습한 EfficientNet + Pseudo-mix 데이터 앙상블
- 평가 지표: **macro-AUC** — 234종 각각 AUC를 평균. 1.0에 가까울수록 좋음.
AUC = 맞는 새와 틀린 새를 얼마나 잘 구분하나 (Slide 7 참고)
- 앙상블 — Perch(72%) + EfficientNet(15%) + Pseudo-mix(6%) + 기타(7%) 가중 합산



소리 → 그림 → 새 이름

6주, 62번 시도, 0.022점 상승

0.912 → 0.934

시작 점수 최고 점수 (+0.022)

(만점 1.0 기준, Public 점수)



● 점수 오른 6번 ● 효과 없거나 떨어진 56번

↑ 여기 6번이 점수를 올렸어요

62

총 시도 횟수

54

Kaggle 제출

12

ralph-x 자동 반복

62번 — 어떤 순서로 뭘 시도했나

1 기초 셋업

시도 1~13

- 자체 파이프라인 3종 실패
Perch CPU 타임아웃 / row_id 형식 / TF 버전
- 공개 노트북 fork → **0.912**

2 포스트프로세싱

시도 14~18

- temperature scaling
- file-level confidence / rank-aware scoring
- → 모두 **하락** (최저 0.904)

3 강한 베이스 탐색

시도 19~28

- 0.928 베이스 노트북 fork
- ONNX Runtime 변환 → 속도 2배
- multi-seed 앙상블 → 타임아웃
- → **0.928**

4 CNN 블렌딩

시도 28~38

- EfficientNet 1-fold blend 8% → **0.929**
- LSE pooling 추론 → 0.922 실패
- Knowledge Distillation → **0.930**
- blend 비중 0.08~0.15 → 모두 동률

5 다양화 시도

시도 39~48

- pseudo-label → 하락 / ProtoSSM → 하락
- HGNetV2 추가 → 하락 / SoftAUC → 동률
- ConvNeXt / prior mask → 동률
- + **Kaggle silent reject 7연속**

6 Multiwindow + Pseudo-mix

시도 49~56 — best 갱신

- EffNet multiwindow fold0 → **0.932**
- pseudo-mix fold0 추가 → **0.933**
- pseudo-mix 4-fold 앙상블 → **0.934 ✓**
- 5-fold → 0.934 동률

7 0.934 천장 돌파 시도

시도 57~62 — 모두 무효 또는 하락

Rank-average fusion
-0.005

ConvNeXt 추가 2연속
Kaggle 타임아웃

EffNet weight 재배분 6연속
모두 0.934 동률

성공한 6번 vs 실패·무효과 56번

통한 것

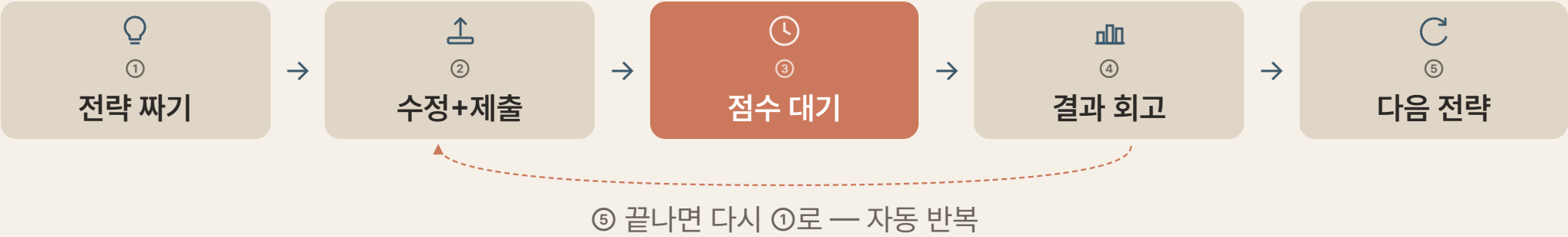
- ✓ **Perch (72%)** — Google 새 소리 전용 AI. 9,736종 사전학습. 점수의 70%+ 담당.
처음부터 이게 핵심. 나머지는 여기에 더하는 구조
- ✓ **EfficientNet 5-fold (15%)** — M1 맥북 MPS로 직접 학습. 5번 다른 조합 평균.
- ✓ **Pseudo-mix (6%)** — Perch 예측값을 실환경 녹음에 붙여 추가학습 → 0.933 → 0.934
돌파
모델 예측값을 '정답'처럼 써서 데이터 늘리는 기법
- ✓ **데이터 증강** — SpecAugment, Mixup
- ✓ **분포 맞는 외부 데이터 선별** — 무차별 추가는 오히려 하락

안 통한 것

- ✗ **Rank-average fusion** → **-0.005 급락**. 확률 크기 정보 손실이 문제.
- ✗ **ConvNeXt (2연속 타임아웃)** — Kaggle이 모델 파일 마운트 거부. 점수 확인 불가.
- ✗ **Knowledge Distillation** — 새 모델 추가해도 효과 0
- ✗ **EffNet weight 재배포 6연속** — 모두 0.934 동률. 같은 계열 내 비율 조정 한계.
- ✗ **학습률 과도** → 발산 / **외부 데이터 무차별** → 분포 어긋남

자동화 — ralph-x 루프

Claude가 혼자 반복하는 흐름



자는 동안 12번 자동으로 반복됐다.
사람은 잠을 자지만 모델은 학습을 멈추지 않는다.

12

자는 동안 자동 반복

54

Kaggle 제출 (절반은 야간 자동)

내 컴퓨터에선 잘 나오는데 제출하면 떨어진다 (CV-LB gap)

CV (Cross-Validation) — 내가 가진 학습 데이터 안에서 나눠 매긴 점수 → **0.95**

LB (Leaderboard) — 캐글에 제출했을 때 받는 점수 → **0.88**

둘 사이 차이를 CV-LB **gap**이라 부른다.

원인: 학습 데이터에 너무 익숙해진 모델 (overfitting = 과적합).

점수 비교



↓ gap = 0.07 — 이게 크면 클수록 외워버린 거

해결책: **dropout**(학습 중 일부 뉴런 무작위 끄기), **augmentation**(데이터 변형해 더 보여주기) — 용어 정리는 **Slide 19** 참고.

※ 예시 수치 (실제 BirdCLEF 결과는 Slide 22 참고)

더 있었던 삽질 — 짧게



학습 데이터 분포 문제

외부 데이터를 잔뜩 더했더니 점수가 오히려 떨어졌다.
양보다 "내 시험 데이터와 비슷한 분포"가 중요했다.

좋은 데이터를 선별하지 않고 무작정 추가하면 모델이 엉뚱한
것을 학습한다.



플랫폼 타임아웃

노트북이 9시간 제한에 걸려 자동 종료됐다.
trial_060, trial_061 — 두 번이 이걸로 실패.

자동화돼도 플랫폼 한계는 사람이 풀어야 한다.



Kaggle 인증 함정

Claude가 임의로 `mcp__kaggle__authorize`를 호출
하면 인증이 깨졌다.

해결: **kaggle.json CLI 방식만** 쓰도록 메모리에 고정.

AI도 도구. 잘못된 도구 호출은 사람이 가드레일로 막는다.

그래도 결과: **0.912 → 0.934**. 삽질은 비용이 아니라 학습이었다.

이런 곳에서 실제로 쓰인다



추천 시스템

유튜브·넷플릭스 다음 영상 추천
쿠팡·무신사 상품 추천
스포티파이 플레이리스트



의료 AI

CT·MRI 이미지로 암 조기 발견
피부 사진으로 피부질환 분류
신약 효능 예측



자연어 처리

ChatGPT 같은 언어 모델
파파고·DeepL 번역
이메일 스팸 필터



금융·보안

카드 사기 거래 실시간 탐지
신용 점수 산정
비정상 로그인 감지



제조·산업

카메라로 불량품 즉시 감지
설비 고장 사전 예측
물류 최적 경로 탐색



환경·과학

새 소리로 생물 다양성 모니터링
위성 이미지로 산불·홍수 감지
기후 변화 시뮬레이션

지금 시작하려면

- ☐ **kaggle.com** 가입 (1분)
- ☐ **Playground Series** 대회 1개 클릭
- ☐ Notebooks 탭 → Most Voted 첫 번째 → **Copy & Edit (fork)**
- ☐ **Run All** → **Submit**. 점수가 뜨면 끝.

📖 첫 질문 추천

"이 노트북 코드가 뭘 하는 건지 처음부터 한국어로 설명해줘."

복사해서 그대로 쓰면 됩니다.

Q&A

무엇이든 물어보세요.

용어가 모호하면 슬라이드 19 (용어 정리)으로 돌아가서 같이 보겠습니다.

—— kangraemin · 2026

Appendix — 참고 링크

공식 사이트

- [kaggle.com](#) — 대회/노트북/데이터셋
- [claude.ai](#) — Claude
- **Playground Series** — [kaggle.com/competitions?category=playground](#)
- `pip install kaggle` → `~/.kaggle/kaggle.json` 인증

주요 도구

- **Claude** — 코드 해설 · 에러 해결 · 전략 제안
- **Kaggle CLI** — `kaggle competitions submit` 등 제출 자동화
- **ralph-x** — 전략 → 제출 → 회고 자동 반복 (12회 야간 자동)

이번 대회

- **BirdCLEF 2026** — [kaggle.com/competitions/birdclef-2026](#)
- ralph-x 자동 루프로 야간 12회 반복
- 이 발표 슬라이드 — [github.com/kangraemin](#)

추천 대회 (초보용)

- **Playground Series** — 매달 새 대회, 초보 친화
- **Titanic** — 머신러닝 입문 표준
- **House Prices** — 첫 회귀 (regression) 입문